

1. 计算机系统漫游

C 语言编译的过程

hello.c 预处理 → hello.s 编译 → hello.o 汇编 → hello 链接

• gcc 预处理 -E, 编译 -S, 汇编 -c, 链接无参数

2. 数据表示

- Dec 转 Bin 分解为 2 的 n 次幂, 1 后面跟 n 个 0
- 字长: 64bit 32bit 代表通用寄存器位数
- 16 进制取反码: 每位用 F 减去

Dec 小数转 Bin 小数(可能无限循环)

乘 2 取整, e.g.: 1.25, 分离小数部分得: 0.25
0.25 * 2 = 0.5 取 0 余 0.5
0.5 * 2 = 1.0 取 1 余 0
从上往下: (1.25)₁₀ = (1.01)₂

端序: 以字节为单位的顺序

e.g.: (i32) 0x1A3CFF12

	小端序	大端序	
低地址	12 FF 3C 1A	1A 3C FF 12	高地址

布尔代数, 分清位运算和逻辑运算

- & / ^ / &: 1 & 1 = 1 0 & ? = 0
- | / ∨: 1 | ? = 1 0 | 0 = 0
- ~ / ~非: 反转
- ^ / ⊕异或: 不相同 ⇒ 1, 相同 ⇒ 0
- 注意: 位运算优先级低于关系运算(><), 高于逻辑运算
- 逻辑运算: &&与, ||或, !非, 按位运算结果非 0 为真(1)0 为假(0)
- 短路运算: 逻辑与 && 左规则则不算右, 逻辑或 || 左真则真不算右

左/右移

- 算术右移: (高位填符号) 0b10010101 >> 4 == 0b11111001
- 逻辑右移: (高位填 0) 0b10010101 >> 4 == 0b00001001
- 注意: 优先级低于算术运算, 高于位运算

负数补码 = 正数的反码(非运算结果) + 1

整数

- 有符号和无符号比较时按无符号比较, 注意 sizeof() 返回 size_t 是无符号机器字长类型
- 短整数和长整数比较按长整数比较(优先)
- 掩码判断第 n 位是否为 1: x & (1 << n) != 0, 注意 n 从 0 开始

表示范围, n 为位数

- 有符号数: -2⁽ⁿ⁻¹⁾ 到 2⁽ⁿ⁻¹⁾-1
- 无符号数: 0 到 2ⁿ - 1

溢出

- 有符号溢出: (正溢)正+正=负, (负溢)负+负=正
- 无符号溢出: 大+大=小
- 加法满足结合律和交换律

IEEE 754 浮点数

- 浮点数转 i32 溢出是未定义行为, 通常返回 TMin(0x80000000)

高地址	s	exp	frac	低地址
f32	31⇒1bit	30-23⇒8bit	22-0⇒23bit	0
f64	63⇒1bit	62-52⇒11bit	51-0⇒52bit	0

$V = (-1)^s * M * 2^E$

- s: 符号位 1 负 0 正
- exp: 阶码(无符号数)
- M: 规格化时 $M = \text{frac} + 1$, 非规格化时 $M = \text{frac}$
- Bias: 2^{k-1} - 1 k 是阶码的位数
- frac: 尾数

规格化数 非规格化数, 表示非常接近 0 的数

- $E = \text{exp} - \text{Bias}$
- $M = \text{frac} + 1$

非规格化数数固定用 1 减的原因: 可以从非规格化值平滑转换到规格化值

特殊值

- inf: exp 全为 1, frac 全为 0, 无穷大, 有符号
- NaN: exp 全为 1, frac 不全为 0, 未定义(如 0 除 0, 无穷减无穷, 负数 sqrt)

手工转换浮点数

e.g.: 将 (10.2)₁₀ 转为 f32 (规格化)
10.2 = (1010.0011...)₂ = (1.0100011... × 2⁽³⁾¹⁰)₂

s = 0, Bias = 127, E = 3 + 127 = 130 = (10000010)₂

- 取 frac 前 23 位: 01000110011001100110011

- 组合: 0 10000010 01000110011001100110011

= 0x41233333, 内存表示: (小端序存储为 33 33 23 41)

向最近偶数舍入

- 舍入位: 最后一位要保留的位的下一位
- 守卫位: 舍入位的前一位
- 粘滞位: 守卫位之后的所有位的逻辑或
- 舍入规则:
 - 若舍入位为 0, 则直接截断
 - 若舍入位为 1, 且守卫位和粘滞位至少有一位为 1, 则向上进 1
 - 若舍入位为 1, 且守卫位和粘滞位均为 0, 则看最后一保留位是奇数还是偶数, 奇数则向上进 1, 偶数则直接截断

间距(步长)

- 最大值间距: 2 的负(frac 位数+Bias)次方
- 最小值间距: 2 的负(1-Bias+frac 位数)次方
- 非规格化间距(恒定): 等于规格化最小值间距

临界值

- 0: 有 +0 和 -0 两种表示, 均为 exp 和 frac 全 0
- 最小正规格化数 exp=1, frac=0: 2ⁿ[-1-Bias]
- 最大非规格化数 exp=0, frac=全 1: (1-2ⁿ[-frac 位数])*2ⁿ[-1-Bias]

上下溢

- 上溢: 超过最大可表示数, exp 计算 ≥ 255, 变为 inf
- 下溢: 低于最小可表示数, exp 和 frac 均为 0 则变为 0, exp 为 0 frac 不为 0 则非规格化数

Unicode

- UTF-8: 变长编码, 1-4 字节, 无大小端序
- UTF-16: 变长编码, 2 或 4 字节, 有大小端序
- UTF-32: 定长编码, 4 字节, 有大小端序

UTF-8 编码规则

范围 U+	1st	2nd	3rd	4th
00-7F	0xxxxxxx			
080-7FF	110xxxxx	10xxxxxx		
0800-FFFF	1110xxxx	10xxxxxx	10xxxxxx	(常用汉字区)
10000-10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

e.g.: 编码 “字”: U+5B57

- 范围: U+0800-U+FFFF, 用 3 字节表示
- 二进制: (0101 1011 0101 0111)
- 使用 3 字节的模板: 1110xxxx 10xxxxxx 10xxxxxx

- 按模补填充: 11100101 10110101 10010111 = E5 BD 97

程序的机器级表示, 汇编(AT&T 语法)

相同的 ?? 寄存器操作的是同一块存储单元, 但位宽不同

寄存器大小

- 字母型寄存器(16 位是原名, 字母不够用 x 补): rax: 64 位, eax: 32 位, ax: 16 位, al: 8 位低, ah: 8 位高
- 数字型寄存器: r12: 64 位, r12d: 32 位, r12w: 16 位, r12b: 8 位

有特殊用途的寄存器

- %rax: 存放函数返回值; 系统调用号传参
- %rsp: 栈指针, 指向当前栈顶位置
- %rbp: 基址指针, 指向当前栈帧底部
- %rip: 指向下一条指令的地址, call 和 ret 会修改它

函数参数传递寄存器

- 64 位程序: 按顺序依次用 %rdi, %rsi, %rdx, %rcx(系统调用使用 %r10), %r8, %r9, 之后参数用栈传递
- 32 位程序: 全部用栈传递(Linux x86-32)

常用指令

- mov %src, %dst: 将 %src 的值复制到 %dst
- push %src: 将 %src 的值压入栈顶, %rsp 会减 8
- pop %dst: 将栈顶值弹出到 %dst, %rsp 会加 8
- lea (%src), %dst: 计算 %src 的地址并存入 %dst, 不访问内存
- call label: 调用函数, 跳转到 label 处执行, 会自动将下一条指令地址压入栈中, 栈会减机器字长(64 位时为 8)
- ret: 从函数返回, 弹出栈顶地址到 %rip
- leave: 销毁栈帧, 等同于 mov %rbp, %rsp 和 pop %rbp
- \$imm: 立即数, 常数值, 如 \$5 表示数字 5
- syscall: 系统调用, 用 %rax 传调用号

条件跳转

标志位

标志位	含义	置 1 时结果满足的条件
ZF	零	结果为 0
SF	负	结果最高位为 1
CF	无符号进位	无符号运算有进位
OF	有符号溢出	有符号运算溢出
PF	偶	最低字节中 1 的个数为偶数

cmp %aaa, %bbb 计算 %bbb - %aaa 但不保存结果, 只更新标志位
无符号: 对 CF 和 ZF 进行逻辑上的组合即可
有符号: SF != OF 表示小于, 相等则 ≥, 以 SF 和 OF 的关系为单位与 ZF 进行逻辑组合

test %aaa, %bbb 类似 and 但不保存结果, 只更新标志位

- 特殊: CF 和 OF 清 0

常用条件跳转指令

指令格式为 j?? label 或地址

第一个 ?: not, 或者空着

第二个 ?:

- 大于: greater(有符号), above(无符号)
- 小于: less(有符号), below(无符号)
- 等于: equal
- 还可以直接判断标志位 jz, jnc 等, jpo = jnp, jpe = jp

e.g.: 有符号不大于 ⇒ jng (not greater) 或者 jle

lea 指令的特殊用法

- lea 指令可以用来进行某些简单算术运算, 效率更高:
lea (%base, %index, scale), %dst 意思是计算 %base + %index * scale 并存入 %dst
- 加法: lea (%src1, %src2, 1), %dst 计算 %src1 + %src2
- 乘法: lea (%src, %src, n), %dst 计算 %src * (n + 1), n 只能是 1、2、4、8
- 地址计算: lea 8(%base), %dst 计算 %base + 8

地址访问

- 直接寻址: mov 0x114514, %dst 将地址 0x114514 处的值复制到 %dst
- 寄存器间接寻址: mov (%rax), %dst 以 %rax 的值作为地址, 将其处值复制到 %dst
- 基址加偏移寻址: mov 8(%rbp), %dst 以 %rbp + 8 作为地址, 将其处值复制到 %dst
- 基址加变址寻址: mov (%rbx, %rcx, 4), %dst 以 %rbx + %rcx * 4 作为地址, 将其处值复制到 %dst, 注意跟 lea 的区别, lea 会直接将计算结果复制, 不访问其处的值
- d(b,i,s): d 只能是立即数, b 任何寄存器, i 除 %rsp 外寄存器, s 只能 1, 2, 4, 8

宽度指令

xxx? 格式的指令 xxxb 8 位, xxxw 16 位, xxxl 32 位(特殊, 64 下会将目标寄存器的 高 32 位清 0), xxxq 64

- movl \$0x12345678, (%rsp) 内存布局:

地址	0x1000	0x1001	0x1002	0x1003
内容	0x78	0x56	0x34	0x12

扩展指令(小宽度复制到大数据)

格式: xxx???, 第一个为扩展方法 s 或 z, 第二三分别为源和目标的宽度

- movswl: 带符号扩展, 32 位目标剩余高位按 16 位源的符号位填充
- movzbl: 0 扩展, 64 位目标剩余高位全部填 0

条件传送 cmov?? %src, %dst, ??同条件跳转, 不支持 8 位

栈操作

- 栈向低地址增长, push 后 %rsp 减 8, pop 后 %rsp 加 8
- 在栈上分配空间: sub \$size, %rsp
- 释放栈空间: add \$size, %rsp

函数调用约定

x86-64

- 调用者保存寄存器: %rax, %rcx, %rdx, %rsi, %rdi, %r8-~%r11, 由调用者负责在调用前保存和调用后恢复, 函数内可以随便用
- 被调用者保存寄存器: %rsp, %rbx, %rbp, %r12-~%r15, 由函数自行保护和恢复, 调用者不负责

被加数: 结果存到哪哪就是被加数

各种计算指令

- add: 加法
- sub: 减法
- imul: 有符号乘法
- mul: 无符号乘法
- neg: 取相反数, 会设置标志位 CF=1(非 0 时)和 OF
- sh?: 逻辑左 / 右移 r(补 0)
- sa?: 算术左 / 右移 r(补符号)
- ro?: 循环左 / 右 r 移

GDB 调试常用命令

- break <func>: 设置断点
- run <args>: 运行程序并传参
- n/s: 单步执行(不/进函数)
- x/<fmt> <addr>: 查看内存, fmt 格式如 8x
- c: 继续执行直到下一个断点
- p <var>: 打印变量值
- i reg: 查看寄存器状态
- i proc map: 查看内存映射
- layout asm: 查看汇编代码布局

结构体对齐

- 成员起始地址 % 成员类型大小 == 0, 成员前填充空字节对齐
- 结构体大小 % 最大成员类型大小 == 0, 结构体末尾填充空字节
- 从大到小排最省空间

操作数

- mov %src, %dst 中 %src 是源操作数, %dst 是目标操作数
- 操作数类型有: 寄存器、立即数、内存引用

攻击

- 栈溢出: 覆盖返回地址, 跳转到恶意代码
- 格式化字符串漏洞: 利用 %n 写任意地址
- 使用后释放: 重复释放或使用已释放内存

防御

- 栈保护(金丝雀): 在栈帧中插入随机值, 检测是否被篡改, --fstack-protector
- 地址空间布局随机化(ASLR): 随机化内存地址
- 数据执行保护(DEP): 标记内存区域为不可执行, 防注入
- 主动检测缓冲区长度, 防止溢出

4. 处理器体系结构, 虚构 Y86

- 超标量 CPU 平均时钟周期小于 1

Y86 ISA 规范 & 阶段流程, 变长指令阴影不占大小

指令	icode	ifun	rA	rB	valC		阶段	操作 SEQ
大小	半	半	半	半	半	8	取指	icode: ifun ← $M_1[PC]$ rA: rB ← $M_1[PC + 1]$ valC ← $M_6[PC + 2]$, valP ← PC + (指令长)
halt	0	0						
nop	1	0						
rrmovq	2	0	rA	rB			译码	valA ← $R[rA]$ (或 $R[\%rsp]$ 若为栈操作) valB ← $R[rB]$ (或 $R[\%rsp]$ 若为栈操作)
irmovq	3	0	F	rB	valC			
rmmovq	4	0	rA	rB	valC		执行	valE ← valB/A/C 计算 或 valB + 8 (栈指针) Set CC
rmrmovq	5	0	rA	rB	valC		访存	valM ← $M_8[valE/A]$ $M_8[valE] ← valA/P$
OPq	6	ifun	rA	rB			写回	R[rA] ← valE/valM R[%rsp] ← valE (若更新 rsp)
j??	7	ifun			valC			
call	8	0			valC			
ret	9	0						
pushq	A	0	rA	F				
popq	B	0	rA	F				
R	rax	rcx	rdx	rbx	rsp	rsi	rdi	
code	0	1	2	3	4	5	6	7
R	r8	r9	r10	r11	r12	r13	r14	无
code	8	9	A	B	C	D	E	F

- valA: 存入内存的数据或弹出的地址
- valB: 基址或当前栈指针
- 条件码: ZF, SF, OF
- valA: 存入内存的数据或弹出的地址
- valB: 基址或当前栈指针
- 条件码: ZF, SF, OF

注意访存阶段不可使用 valB, 写回只能用 valE/M; valB 永远是被操作数, 非空执行

阶段 valB 必须出现

- 程序状态: AOK: 正常, HLT: 停机, ADR: 非法地址, INS: 非法指令

指令执行步骤

- 取指 F: 从 (valP/C/M) 选择 PC 从内存取出指令到寄存器, 计算下一条指令地址 (valP), 取出立即数 valC
- 译码 D: 解析指令, 最多读取两个寄存器 (valA, valB) 的值
- 执行 E: 在算术逻辑单元 (ALU) 中执行指令计算结果 valE, 更新 CC
- 访存 M: 根据 valE (此时为地址) 写入内存或读取数据 (valM)
- 写回 W: 将结果 (valE 或 valM) 写回寄存器
- 更新 PC: 根据指令类型和结果选择下一条指令地址 (与取值同时进行)

流水线冒险

- 数据冒险: 下游指令依赖上游指令结果, 使用旁路转发, 或直接暂停
 - valE 冒险: 直接转发 valE 到下游指令的 D 阶段
 - 加載使用冒险 (valM): 在 E 插入 1 个气泡, 之后转发
- 控制冒险: 分支指令不知道跳转目标, 使用分支预测或流水线停顿解决
 - 分支预测错误: E 时发现错误, 用 2 个气泡覆盖 F 和 D 的 2 个错误指令, 从正确目标重新取指
 - ret: 在 M, E, D 阶段插入 3 个气泡, 获得 valM 后 F 阶段选择其作为 PC

时序/组合逻辑

- 时序逻辑: 有记忆功能, 写操作使用
- 组合逻辑: 无记忆功能, 读操作使用

5. 优化程序性能

C 语言多维数组按行优先存储, arr[100][50] 的布局为 100 个 有 50 个元素的行依次存储, 因此优先访问最后的 [] 效率最高

优化方法

一般有用的优化

- 代码移动: 将重复计算的代码移出循环
- 减少函数调用开销: static inline 内联函数, 减少调用返回开销
- 共享子表达式消除: 使用指针代替索引, img[h][w].ptr=&img[0][0], img[i][j] 改为 *(ptr+i*w+j)

面向编译器优化

- 消除内联别名使用: 使用 restrict 声明指针不重叠或局部变量存中间结果算完再写回
- 内联函数避免编译器因无法分析副作用而不进行优化

超标量 CPU 优化

- 带累加器循环展开: 减少分支预测开销, 增加指令级并行(打破顺序执行限制)
- 多个累积变量: 利用更多运算单元同时工作

向量处理器优化

- 向量化: SIMD 指令 (AVX 等), 一次处理多个数据
- 数据对齐: 确保向向量宽度对齐, 加速向向量加载和存储

Cache 优化

- 优先访问多维数组的行: 利用空间局部性
- 分块: 提高局部元素的时间局部性, 避免缓存抖动提高命中率

6. 存储器层次结构

易失性存储器

- SRAM: 只要有电就能保存数据, 无需刷新, 抗干扰
- DRAM: 会漏电, 需要定期刷新保持数据, 不稳定

非易失性存储器

- HDD: 延迟=寻道+旋转+传送, 其中寻道时间最慢, 可直接覆盖写入
- SSD: 写入单位是页, 擦除单位是块, 只能从 1 变 0, 不可覆盖写入, 因此写入前需先擦除成 1

存储器山

- stride 轴: 工作集大小, 代表时间局部性
- stride 轴: 访问步长, 代表空间局部性
- 纵轴: 吞吐量 (MB/s)

高速缓存

这种结构成为E路组相联 Cache，共有S组，若S=1则为**全相联**，若E=1则为**直接映射**，block大小称为块大小，现代CPU采用**组相联**

- 冷/强制不命中：访问的块不在 Cache 中
- 容量不命中：容量不足必须替换已有块

访问 Cache 使用**物理地址**

- 冲突不命中：映射到同一组的块过多，导致替换
 - C: 总缓存大小，C=S*E*B
 - S: 组数
 - E: 每组行数(路数)
 - B: 块大小(字节数),Intel 为 64B
 - b: 块内偏移位数: log2(B)
 - s: 组索引位数: log2(S)
- 不命中率：不命中次数/总访问次数，总访问次数=写次数+读次数，不命中次数为从内存向 Cache 调入数据的次数
 - 平均存取时间：不命中率 * 不命中时间 + 命中率 * 命中时间

写策略

写命中

- 全写：立即更新 Cache 同时写入主存，效率低但数据一致
 - 写回：只更新 Cache，只有驱逐时才写入主存，效率高，需要新增脏位，为 1 代表比主存新，为 0 直接丢弃；效率高，多核逻辑复杂，有一致性问题
- 非写分配：直接写入主存，常与全写搭配

写不命中

- 写分配：先读入 Cache 再写，适合空间局部性好的场景，常与写回搭配
- 非写分配：直接写入主存，常与全写搭配

7. 链接

- 可重定位目标文件和共享库代码地址从 0x0 开始
- 可执行文件地址从 0x400000 开始
- call 和各种 jump 实际存储的地址字节都是相对 %rip 的偏移量，%rip + 偏移量 = 目标地址
- 加载器: ld-linux.so*, 负责加载共享库，处理动态链接

可重定位目标文件 (.o)				可执行目标文件			
位置/大小	ELF 头	段		ELF 头	第一条 addr		
代码/只读数据格式串	text	代码		段头部表	内存映射		
跳转表	.rodata			init	init	只读段开始	
data 初始化为 0 bss 未初始化/0	.data	数据	映射只读段	text	代码段		
	.bss		映射读写段	.rodata			
重定位条目	.rel.text	链接用	映射读写段	.data	数据段		
	.rel.data			.bss			
	.debug/.line	调试/符号表	不加载内容	.symtab	符号/调试区		
	.symtab			.debug			
	.strtab			.strtab			
	节头部表			节头部表			

重定位条目

- 外部函数：在 .rel.text 中，有符号名和重定位类型
- 全局变量：若全局使用则在 .rel.data 中，若局部使用则 .rel.text
- R_X86_64_32: 绝对重定位，32 位地址，符号地址
- R_X86_64_PC32: 相对 PC 的 32 位重定位，符号地址-%rip
- R_X86_64_PLT32: 函数的相对 PC 重定位，符号地址-%rip

链接命令

- .a 被依赖的放在后面，循环依赖需要多次填写
- .o 无顺序，同名只出现一次

链接流程

- 符号解析: 处理外部符号，找到符号定义位置
- 符号重定位: 修改代码和数据中的地址

静态 & 动态链接

- 生成静态库: ar rcs libname.a file1.o file2.o ...
- 动态链接库必须地址无关才能加载到内存的任何位置
- GOT: 全局偏移表，用于访问全局变量，加载器在运行时修改 GOT 条目为真实地址
- PLT & 延迟绑定: 用于调用外部函数，第一次调用时才会修改 GOT 进行地址解析，后续直接通过 GOT 跳转

节和段

- 代码段 r-x: .text+.rodata+.init+ELF 头
- .text: 代码节，存放程序指令
- .rodata: 只读数据，全局(static) const、字符串常量、跳转表
- .init: 初始化函数，如 .init，程序入口前执行
- 数据段 rw-: .data+.bss+.got
- .data: **已初始化**的非 0 全局和静态变量
- .bss: **未初始化**或初始化为 0 的全局和静态变量，只记录大小，运行时才分配空间并清零
- 栈段 rw-: 栈空间
- 堆段 rw-: 堆空间

符号解析

- 全局符号: 当前模块定义，并能被其他模块引用的符号
 - 强符号: 函数和已初始化的全局变量，不可重复定义
 - 弱符号: 未初始化的全局变量，可重复或被强符号覆盖，如果有多个弱符号无强符号则任选一个，只在链接前存在，现代编译器默认禁止隐式弱符号
- 局部符号(Local): 仅在当前文件可见，static 修饰，并非局部变量
- 外部符号: 由其他模块定义被当前模块引用的全局符号，extern 修饰，静态链接后消失

库打桩发生时机

- 编译时：使用宏定义在预处理阶段替换函数调用
- 静态链接时：定义 __wrap_<func> 函数，链接时用 -Wl,-wrap=<func>
- 动态链接产物运行时：用 LD_PRELOAD 环境变量指定替代共享库

8. 异常控制流

- 异常处理程序拥有系统资源的完全访问权限
- 异常发生时硬件通过**异常表**跳转到对应的内核异常处理子程序
- 0-31 号为硬件/内核异常，32-255 号为系统调用/外部 IO 信号/用户自定义异常

异常类型

异常类型	触发原因	同/异步	返回行为
中断	来自 I/O 设备请求	异步	继续下一条指令
陷阱	程序有意(syscall)	同步	继续下一条指令
故障	潜在可恢复错误(整/0)	同步	可能重试或终止程序
终止	不可恢复的错误	同步	不返回

(非)本地跳转，都不是系统调用

- 本地跳转: 同一函数内跳转，使用 if、for、goto 等控制结构实现

- 非本地跳转: 跨函数跳转，setjmp 保存，longjmp 跳转并恢复，只会恢复寄存器和栈指针

进程

- 子进程必须由父进程回收，否则成为僵尸进程
- errno: 记录错误码的外部全局变量，系统调用失败时设置，使用 strerror(errno) 获得错误原因
- 僵尸进程：子进程结束后父进程未调用 wait(pid) 回收，保留进程表项但资源已经释放
- 孤儿进程：父进程结束后子进程被 init 收养，若子进程结束则孙进程仍然被 init 收养

调度

- 进程上下文包括：寄存器，一级页表**指针**(寄存器)，进程表，文件表(描述符)，不包含全局变量等内存中的数据
- 时间片轮转调度: 每个进程分配固定时间片，时间片用完则切换到下一个进程
- 上下文切换: 1)保存当前进程的上下文，2)恢复某个先前被抢占的进程被保存的上下文，3)将控制传递给这个新恢复的进程

pid_t fork()

创建父进程的副本作为子进程，调用一次父子各返回一次，子进程返回 0，父进程返回子进程 PID，继承文件描述符

int execev("filename", *argv[], *envp[]) 不返回

- filename: 可执行文件路径
- argv[]: 参数数组，argv[0] 为程序名
- envp[]: 环境变量数组，可传全局变量 environ

用新程序替换当前进程映像，加载新程序到当前进程空间并运行，保留 PID 和文件描述符

int waitpid(pid, int *status, int options)

- wait(): 简化版，等同于 waitpid(-1, status, 0)
- pid: 待回收的子进程 PID，-1 表示回收任意子进程
- status: 返回子进程退出状态，调用者传入缓冲区
- options: 怎么等待，0 表示阻塞等待，其他选项:

pid_t getpid()

获取当前进程的 PID

pid_t getppr()

获取当前进程所属的进程组 ID

pid_t getppid()

获取当前进程的父进程 PID

信号使用 kill -? <pid> 发送

- SIGINT(2): 中断，Ctrl+C
- SIGKILL(9): 强制终止，无法捕获和忽略
- SIGSEGV(11): 无效内存访问，段错误
- SIGCHLD(17): 子进程状态改变通知父进程
- SIGFPE(8): 浮点异常，但整数除 0 也有
- SIGALRM(14): 定时器到期
- SIGSTOP(19): 暂停进程，无法捕获和忽略，Ctrl+Z
- SIGCONT(18): 继续执行暂停的进程
- SIGTERM(15): 默认行为为终止进程，可捕获和忽略

信号处理

- signal(signalum, handler): 注册信号处理函数,其中 SIG_IGN 忽略,SIG_DFL 恢复默认
- 信号处理程序可被其他信号处理程序中断

信号阻塞

- sigprocmask(how, \set, \oldset): 阻塞或解除阻塞 set 中的信号
 - how: SIG_BLOCK 阻塞，SIG_UNBLOCK 解除阻塞，SIG_SETMASK 设置新的阻塞集
 - set: 指定要阻塞/解除的信号集
 - oldset: 可选，返回之前的阻塞集
- 还有其他函数 "empty", "fill", "add", "del", "ismember" 等

异步信号安全函数

- 信号处理程序中只能调用异步信号安全函数，否则可能引发死锁或数据损坏
- 系统调用都是异步信号安全的
- 纯 C 库的都不安全，带缓冲区，如 printf、malloc 等

等待信号，循环是一种浪费

- sigsuspend(*mask): 替换当前阻塞信号集为 mask 后挂起等待信号，返回后恢复原阻塞集

TinyShell

- shell 定义: 命令行解释器，提供用户与操作系统交互的接口，一个交互型的应用级程序，它代表用户运行其他程序
- 泰山服务器 ARM 架构上 char 为无符号类型，需要改为 signed char 强制指定，否则 getopt 返回 -1 时会被解析为 255

运行命令的流程

- 判断是否为内置命令，若是则直接执行
- 否则 fork() 创建子进程
- 在子进程中用 execev() 加载执行程序
- 父进程用 waitpid() 等待子进程结束并回收
- 子进程调用 exit(0) 退出并向 shell 发送 SIGCHLD 信号
- 父进程捕获 SIGCHLD 信号并回收子进程
- shell 回到命令提示符，等待输入

9. 虚拟内存

内核	内核代码 & 数据
	进程相关数据结构
进程	环境变量
	命令行参数 argv/c
虚拟内存	用户栈 ↓ %rsp 减小
	↑ 共享库内存映射
内存	↑ brk 增大
	运行时堆
数据段	数据段 .bss
	数据段 .data
代码段	代码段 .text 等

页

- 常见页大小 4KB，大小为 2^4VPO 位值
- 常见页对齐: 地址低 12 位为 0(x86-64)

页表

- 每个页表通常有 512 个页表项；一个 PTE 占 8B，一个 PTE 指向一个页或下一级页表，独立页项数 2^4VPN 位值
- 每个进程有独立页表，保护进程地址空间
- 页表项 PTE 包含 PPN(12-51 位)和权限位，如 valid(0 位)、read、write、exec 等
- MMU 将虚拟地址转为页表项地址 PTEA，查页表找到 PTE 得到物理页号 PPN，再组合成物理地址 PA

虚拟地址结构

高位	VPN	VPO	低位
页号	页内偏移		地址

CPU 访问虚拟内存流程

- CPU 用虚拟地址 VA 访问 MMU
- MMU 将 VA 分为 VPN 和 VPO
 - 若 VPN 在 TLB 中命中，取出 PPN，组合 PA = PPN cat VPO
 - 若 VPN 在 TLB 中未命中，计算 PTEA=页表基址+VPN*PTE 大小，访问页表得到 PTE
 - 若 PTE 有效，取出 PPN，组合 PA = PPN cat VPO
 - 若 PTE 无效，触发缺页中断，从磁盘调入页面到内存后重试
- MMU 通过 PA 访问内存并将数据返回 CPU

TLB(快表)

- 高速缓存页表条目，存储最近使用的 VPN 到 PTE 映射
- TLB 命中则快速转换地址，未命中则访问页表
- Intel 的 TLB 每行 Block 存储 8 字节

缺页中断，属于 Fault 故障，并非“中断”

判断合法性->选择牺牲性页->调入目标页->更新页表->重新执行指令

多级页表，现代常用 4 级页表

- 多级页表中，未使用的物理空间无需分配页表，多级页表中原虚拟地址的 VPN 被划分为每级页表的 VPN

级	项数	项索引大小	总索引大小
1	512	4KB	512*4KB=2MB
2	512	2MB	512*2MB=1GB
3	512	1GB	512*1GB=512GB
4	512	512GB	512*512GB=256TB

- 级别越低越“靠近”物理内存
- 原理不是用更少页表管理更多内存，而是稀疏分配节省内存

- 一张 4 级页表管理 256TB 虚拟内存，低级页表稀疏分配，无需占满

内存映射

- 磁盘上的普通文件：比如可执行文件、数据文件
- 匿名文件：由内核创建的全 0“文件”，用于 .bss，堆和栈，映射请求 0 的页

vm_area_struct 链表，简称 VMA，它们构成 mm_struct

- mm_struct: 整个进程的虚拟内存空间
 - pgd: 一级页表指针
- vm_start 和 vm_end: 区域的起始和结束虚拟地址
- vm_flags: 保护权限，如 VM_READ, VM_WRITE, VM_EXEC
- vm_prot: 读写执行权限

用户级内存映射 mmap(start, len, prot, flags, fd, offset)

- start: 虚拟内存起始地址，NULL 表示由内核选择地址
- len: 映射长度，按页对齐
- prot: 保护权限，如 PROT_READ, PROT_WRITE, PROT_EXEC
- flags: 映射类型，如 MAP_PRIVATE, MAP_SHARED, MAP_ANONYMOUS
- fd: 文件描述符，MAP_ANONYMOUS 时忽略
- offset: 文件偏移，MAP_ANONYMOUS 时忽略
- 返回指向映射区域起始的指针

共享对象

共享的数据在修改时使用写时复制 COW，fork() 函数同样使用 COW，只有修改时才复制页面

fork() 实现，创建当前进程副本

- 创建当前进程的 mm_struct, vm_area_struct 和页表的副本，为子进程分配 PID
- 将父子进程的每个页都标记为只读
- 将父子进程的每个 vm_area_struct 都标记为私有的 COW
- 在子进程中返回时，子进程拥有与 fork 调用时父进程相同的虚拟内存
- 子进程之后的写操作通过 COW 创建新页

execve() 实现，替换当前进程

- 删除已存在的用户区域 vm_area_struct
- 映射私有区域创建新的 vm_area_struct，私有 COW，将目标文件的 .text, .data 映射到代码和数据区，将 .bss 和堆栈映射到匿名文件；堆栈初始长度 0
- 映射共享区域用动态链接器将共享对象映射到新进程的共享区域，并对可重定位的代码和数据进行重定位
- 在栈中建立程序参数和环境变量
- 设置程序计数器设置 PC 指向程序入口 _start，返回用户态运行新程序
- 运行中通过缺页中断处理程序按需加载代码或数据页，更新页表
- 进程终止后向父进程发送 SIGCHLD 信号

不知道什么了！就放点定型文吧

在终端中的命令行运行显示 "Hello World" 的执行程序 hello，结合进程创建、加载、缺页中断、内存访问(虚存)……等等，论述 hello 是怎么一步步执行的。答：进程创建: fork, 1 分 程序加载执行: execev、动态链接、mmap 等虚拟地址空间, 2 分 执行指令: 代码段缺页中断, 恢复运行, 2 分 (TLB、Cache、RAM) 访问数据: 数据段缺页中断, 恢复运行, 2 分 显示输出: C 标准函数调用 printf、堆栈段访问, 系统调用 write, 系统级 IO, 显示 RAM 等, 2 分 进程终止: 1 分, 释放。

什么是共享库(动态链接库)? 简述动态链接的实现方法。答：共享库(动态链接库)是一个 .so 的目标模块(elf 文件)，在运行或加载时，由动态链接器程序加载到任意的内存地址，并和一个和内存中的程序(如当前可执行目标文件)动态完全连接为一个可执行程序。使用它可节省内存与硬盘空间，方便软件的更新升级。如标准 C 库 libc.so。1 分 2 分 加载时动态链接: 应用程序第一次加载和运行时，通过 ld-linux.so 动态链接器重定位动态库的代码和数据到某个内存段，再重定位当前应用程序中对共享库定义的符号的引用，然后将控制传递给应用程序(此后共享库位置固定了并不变)。2 分 运行时动态链接: 在程序执行过程中，通过 dlopen/dlsym 函数加载动态链接共享库，实现符号重定位，通过 dclose 卸载动态库。

(0)Linux 系统中，Shell 是一个交互型应用级程序，代表用户运行其他程序(是命令行解释器，以用户态方式运行的终端进程)。其基本功能是解释并运行用户的指令，重复如下处理过程: (1)终端进程读取用户由键盘输入的命令行。(2)分析命令行字符串，获取命令行参数，并构造传递给 execev 的 argv 向量 (3)检查第一个(首个、第 0 个)命令行参数是否是一个内置的 shell 命令 (3)如果不是内部命令，调用 fork() 创建新进程/子进程 (4)在子进程中，用步骤 2 获取的参数，调用 execev() 执行指定程序。(5)如果用用户没要求后台运行(命令末尾没有 &) 否则 shell 使用 waitpid (或 wait...) 等待作业终止后返回。(6)如果用用户要求后台运行(如果命令末尾有 &)，则 shell 返回;